

VitalRisk AI — ADR 002

Metodología CRISP-ML(Q): Diseño de Notebooks y Pipeline ETL

EQUIPO 326

FASE 1 · SPRINT 2

13 JUNIO 2026

ESTADO: PROPUESTO

1. Contexto y propósito de este documento

Este ADR 002 complementa el ADR 001 (Arquitectura Base, 1 junio 2026) y documenta las decisiones metodológicas tomadas el 13 de junio de 2026 para la Fase 1, Sprint 2. Define cómo se aplicará CRISP-ML(Q) al proyecto VitalRisk AI, estableciendo qué trabajo se realiza en Jupyter Notebooks (exploración científica) y qué trabajo se encapsula en Scripts Python ETL (pipeline productivo reproducible).

ADR 001 ya decidió

FastAPI + PostgreSQL/PostGIS como stack tecnológico.
Docker para contenerización. Esquema de 7 tablas en la BD.
SIVIGILA + IDEAM + ECV + DIVIPOLA como fuentes de datos.

Este ADR 002 decide

Cómo se aplica CRISP-ML(Q) paso a paso. Qué análisis van en Notebooks y cuáles en Scripts ETL. El orden de carga de las 7 tablas. Cómo se maneja el período COVID en el pipeline.

2. Decisión central: Notebooks para ciencia, Scripts para ingeniería

La práctica estándar de la industria para proyectos de Machine Learning es separar el trabajo exploratorio del trabajo productivo. VitalRisk AI adoptará esta separación:

	Jupyter Notebooks	Scripts Python ETL
¿Qué es?	Entorno interactivo para explorar, visualizar y entender los datos	Código limpio, modular y reproducible que corre en el pipeline de producción
¿Quién lo ve?	El equipo de datos + los jueces del concurso (evidencia del proceso)	El sistema en producción (FastAPI lo llama, Docker lo ejecuta)
¿Cuándo se usa?	Primera vez que se explora un dataset; análisis de correlación; visualizaciones	Cuando la lógica ya está probada y debe correr automáticamente
¿Qué NO hace?	No carga datos en producción; no define la lógica final del modelo	No hace análisis visual; no requiere supervisión humana para correr
Entregable	Archivos .ipynb en /notebooks/ con outputs guardados (gráficas + hallazgos)	Archivos .py en /backend/etl/ sin outputs — solo lógica pura

3. CRISP-ML(Q) aplicado a VitalRisk AI — Los 9 pasos

CRISP-ML(Q) define 6 fases estándar. En VitalRisk AI cada fase se desglosa en pasos concretos con entregables definidos. Los pasos 1-8 se desarrollan en Notebooks y luego se encapsulan en Scripts. El paso 9 es el despliegue en producción.

FASE 1 — Comprensión del Negocio y los Datos | Responsable: Product Owner + Data Scientist | Sprint: Semana 1

Paso 1

Comprensión del negocio — Definir el problema en términos de ML

■ **En el Notebook**

- Definir KPIs del modelo: RMSE objetivo < 15% del naive baseline
- Documentar la pregunta: '¿Cuántos casos IRA habrá en municipio X la semana t+1?'
- Listar variables objetivo (casos_ira_total) y features candidatas
- Identificar restricciones: datos solo hasta 2023, municipios Valle de Aburrá

■ **En el Script ETL**

- No genera script — solo genera documentación (este ADR + el ADR001)
- Los KPIs definidos aquí se convierten en criterios de aceptación de HU14

Paso 2

Exploración inicial de datos — Entender qué contiene cada fuente antes de limpiarla

■ **En el Notebook**

- 01_exploracion_sivigila.ipynb: columnas, dtypes, valores únicos, nulos
- 02_exploracion_calidad_aire.ipynb: resolución temporal real de PM2.5
- 03_exploracion_ecv.ipynb: confirmar 10 municipios Valle de Aburrá presentes
- 04_exploracion_metrosalud.ipynb: cobertura temporal 2018-2023
- Responder: ¿el código DANE viene en 3 o 5 dígitos en cada fuente?

■ **En el Script ETL**

- extract_data.py: descarga de los 10 datasets con paginación SODA2
- Guarda en /data/raw/ con timestamp (criterio HU5)
- Genera log de: filas descargadas, columnas disponibles, status HTTP

FASE 2 — Ingeniería de Datos (ETL) | Responsable: Data Engineer + Data Scientist | Sprint: Semanas 1-2

Paso 3

Limpieza y normalización — Estandarizar para que todos los datasets hablen el mismo idioma

■ **En el Notebook**

- 05_limpieza_sivigila.ipynb: verificar que cod_mun_o sea DIVIPOLA de 5 dígitos
- Detectar % de nulos por columna — gráfica de mapa de calor de nulos
- Confirmar rango de fechas real (¿llega hasta 2023 o solo 2022?)
- Verificar que semana_epidemiologica sea 1-52 sin valores atípicos
- Validar que periodo_pandemia = True solo para mar-2020 a dic-2021

■ **En el Script ETL**

- clean_data.py: fechas a ISO 8601, columnas a snake_case
- Imputación de nulos según regla definida en el notebook
- Agregar columna periodo_pandemia (criterio HU6)
- Filtrar solo registros 2018-2023
- Guardar en /data/processed/clean_*.csv

Paso 4

Cruce espacial y temporal — Construir la Tabla Maestra que une salud + ambiente + contexto

■ En el Notebook

- 06_merge_datos.ipynb: probar el JOIN (codigo_dane, anio, semana_epi)
- Verificar cobertura: ¿cuántos municipios quedan con datos en ambas fuentes?
- Detectar municipios sin estación IDEAM propia (requieren interpolación)
- Validar: al menos 80% de municipios del Valle de Aburrá con PM2.5 asignado
- Calcular tasa_ira_100k = (casos / poblacion_anio) * 100000 — verificar rango

■ En el Script ETL

- merge_data.py: JOIN (codigo_dane, anio, semana_epi) vectorizado
- Interpolación por estación más cercana para municipios sin cobertura
- Calcular tasa_ira_100k usando dim_poblacion_anual (año correcto)
- Resultado: /data/processed/tabla_maestra.csv (criterio HU7)

Paso 5

Carga en base de datos — Persistir la Tabla Maestra en PostgreSQL/PostGIS

■ En el Notebook

- 07_validacion_pre_carga.ipynb: verificar todos los codigo_dane existen en dim_municipios
- Contar registros esperados vs registros reales post-carga
- Ejecutar query de verificación: SELECT COUNT(*) por municipio y año

■ En el Script ETL

- load_to_db.py: carga con SQLAlchemy + GeoAlchemy2
- Upsert idempotente — ON CONFLICT DO UPDATE (criterio HU8)
- Log de registros insertados vs actualizados
- Orden de carga: dim_municipios → dim_poblacion_anual → dim_estaciones_aire → facts

FASE 3 — Ingeniería de Modelos ML | Responsable: Data Scientist | Sprint: Semanas 3-4

Paso 6

EDA y selección de features — Entender las correlaciones y elegir qué entra al modelo

■ En el Notebook

- 08_eda_correlaciones.ipynb: matriz de correlación Pearson + Spearman
- Visualizar series temporales de PM2.5 vs casos IRA por municipio
- Test de estacionariedad ADF sobre casos_ira_total (requerido para SARIMAX baseline)
- Winsorización: detectar percentiles 1% y 99% de PM2.5 (HU10)
- Seleccionar features con p-value < 0.05 (criterio HU9)
- Marcar variables redundantes (correlación > 0.90 entre sí)

■ En el Script ETL

- feature_engineering.py: winsorización, creación de lag variables (HU12)
- Lag 1 y Lag 2 de PM2.5; Lag 1 de casos_ira
- Split temporal: Train 2018-2021 / Validación 2022 / Test 2023
- Verificar ausencia de data leakage temporal (criterio HU12)

Paso 7

Entrenamiento del modelo — Entrenar XGBoost con las features seleccionadas

■ En el Notebook

- 09_entrenamiento_xgboost.ipynb: entrenar con hiperparámetros base
- Graficar curvas de aprendizaje (Train vs Validación)
- Extraer Feature Importance — graficar top 10 variables más importantes
- Comparar contra naive baseline (media histórica por semana epidemiológica)
- Verificar que RMSE XGBoost < RMSE baseline en al menos 15% (criterio HU14)

■ En el Script ETL

- train_model.py: entrenamiento XGBoost con parámetros validados en notebook
- Guardar artefacto del modelo en /models/xgboost_vitalrisk.pkl
- Guardar reporte de métricas en /docs/models/metrics_report.json (HU14)

Paso 8

Evaluación y motor de alertas — Validar el modelo y generar el sistema de alertas

■ En el Notebook

- 10_evaluacion_modelo.ipynb: calcular RMSE y MAE sobre el Test set (2023)
- Comparar predicciones vs valores reales — gráfica de serie temporal
- Validar con datos de Metrosalud: ¿predijo picos que Metrosalud registró?
- Definir umbral de alerta con base en distribución de percentiles (HU15)
- Calcular IPT para cada municipio y verificar rango 0-100 (HU11)

■ En el Script ETL

- alert_engine.py: lógica de alertas VERDE/NARANJA/ROJA (HU15)
- ipt_calculator.py: cálculo vectorizado del IPT con clipping (HU11)
- Insertar alertas en tabla alertas_territoriales vía SQLAlchemy
- Exportar Golden Path Dataset para la demo (HU24): semana crítica de 2023

FASES 4-6 — Evaluación, Despliegue y Monitoreo | Responsable: Full Stack + DevOps | Mes 3

Paso 9

Despliegue y monitoreo en producción — Integrar el modelo en FastAPI y desplegar en la nube

■ En el Notebook

- 11_smoke_testing.ipynb: probar la API con datos reales post-despliegue
- Verificar tiempo de respuesta del endpoint `/api/v1/mapa/riesgo < 2s`
- Golden Path test: cargar `seed_demo_data.sql` y verificar que el mapa se renderiza

■ En el Script ETL

- main.py: endpoints HU16 con inferencia del modelo cargado desde .pkl
- Docker Compose: tres contenedores (db + api + web) en red interna
- CI/CD: push a main → deploy automático en Render/Railway (HU21)
- Monitoreo: log de predicciones vs valores reales para detectar degradación

4. Orden de carga obligatorio de las tablas (dependencias FK)

Las Foreign Keys del esquema imponen un orden estricto de carga. Intentar cargar en orden incorrecto genera errores de integridad referencial que bloquean el pipeline silenciosamente.

#	Tabla	Fuente de datos	Debe cargarse antes de...	HU
1	dim_municipios	DANE DIVIPOLA + GeoJSON IGAC + ECV 2023	Todas las demás tablas	HU5+HU6
2	dim_poblacion_anual	DANE — Proyecciones CNPV 2018	fact_riesgo_territorial (tasa_ira_100k)	HU5+HU6
3	dim_estaciones_aire	IDEAM dataset 57sv-p2fu	fact_calidad_aire	HU5+HU6
4	fact_calidad_aire	SIATA (pendiente) + 4 datasets IDEAM	fact_riesgo_territorial	HU7+HU8
5	fact_eventos_ira	SIVIGILA 2018-2023 (6 datasets anuales)	fact_riesgo_territorial	HU7+HU8
6	fact_riesgo_territorial	JOIN entre calidad_aire + eventos_ira + municipios	alertas_territoriales	HU7+HU8
7	alertas_territoriales	Generada por el motor XGBoost	—	HU15

5. Decisión arquitectónica: manejo del período COVID-19

Los datos SIVIGILA del período marzo 2020 — diciembre 2021 presentan una ruptura estructural por la pandemia. Esta decisión documenta cómo se maneja en el pipeline para que el modelo no aprenda patrones incorrectos.

Decisión	Implementación	Justificación
Crear columna periodo_pandemia	En clean_data.py: periodo_pandemia = True si fecha entre 2020-03-01 y 2021-12-31	Permite al modelo distinguir comportamiento pandémico del comportamiento normal
NO descartar los datos COVID	Los registros se conservan con el flag, no se eliminan	Cinco años de datos con pandemia siguen siendo más ricos que cuatro años sin ella
Incluir como feature en XGBoost	periodo_pandemia entra como variable binaria (0/1) en el Feature Engineering (HU12)	Le enseña al modelo que existió una perturbación externa de magnitud extraordinaria
Split temporal respeta la pandemia	Train: 2018-2021 (incluye pandemia con flag); Val: 2022; Test: 2023	Evita que el modelo sea evaluado solo en datos post-pandemia sin haber aprendido ese patrón

6. Estructura de repositorio resultante

Carpeta	Contenido	Quién la mantiene
/notebooks/	01 a 11: notebooks de exploración y entrenamiento (.ipynb con outputs guardados)	Data Scientist
/backend/etl/	extract_data.py · clean_data.py · merge_data.py · load_to_db.py · feature_engineering.py · train_model.py · alert_engine.py · ipt_calculator.py	Data Engineer + Data Scientist
/models/	xgboost_vitalrisk.pkl — artefacto del modelo entrenado (en .gitignore)	Data Scientist
/data/raw/	CSVs descargados con timestamp (en .gitignore — no van al repo)	Script automático
/data/processed/	clean_*.csv · tabla_maestra.csv (en .gitignore)	Script automático
/docs/models/	metrics_report.json · threshold_justification.md · ADR001 · ADR002	Todo el equipo
/db/	init.sql v2 · seed_demo_data.sql (HU24)	Data Engineer
/backend/tests/	test_health.py · test_ipt.py · test_integracion_backend.py (HU19)	QA / Todo el equipo

7. Consecuencias de esta decisión

✓ Positivas - Los notebooks sirven como evidencia del proceso científico ante el jurado (HU23) - El pipeline ETL es reproducible en cualquier máquina con docker-compose up - La separación Notebook/Script reduce el riesgo de tener lógica de negocio enterrada en celdas no ejecutadas - El orden de carga de tablas documenta las dependencias explícitamente - El manejo del COVID queda trazable desde el dato crudo hasta la feature del modelo	■ Riesgos y mitigaciones - Riesgo: que el notebook y el script diverjan (mitigación: code review antes de cerrar cada HU) - Riesgo: que los 6 datasets SIVIGILA tengan columnas con nombres diferentes entre años - Mitigación: en el notebook de exploración se verifica la columna a columna antes de escribir el script - Riesgo: que el SIATA no responda antes del Sprint 3 (mitigación: pipeline funciona con IDEAM)
---	---

